

Revision — 2.1.3 Thinking Procedurally

OCR H446 — one-page revision summary

Must know

- **Thinking procedurally** = breaking a problem into an **ordered sequence of steps** and **sub-procedures** (decomposition), then arranging them in the correct order.
- **Decomposition** splits a large problem into smaller, more manageable sub-problems, each solved by its own sub-procedure/subroutine.
- Each sub-procedure should do **one clear job**, making the solution easier to design, test, reuse and maintain.
- **Order matters**: some steps depend on the **output of earlier steps** and must run after them; state which dependencies fix the sequence.
- Sub-procedures can be **reused** (called from several places) and combined to build the whole solution — this links to reusable components (2.1.2).
- Contrast with **abstraction** (2.1.1): abstraction *removes* detail; decomposition/procedural thinking *breaks the problem into ordered sub-steps*.

Key terms

Term	Definition
Thinking procedurally	Breaking a problem into an ordered sequence of steps/sub-procedures
Decomposition	Splitting a problem into smaller sub-problems
Sub-procedure / subroutine	A named block of code doing one part of the task
Sequence	The order in which steps/procedures are carried out
Dependency	When one step needs the output of an earlier step
Modularity	Building a solution from separate, self-contained parts
Parameter	A value passed into a sub-procedure to make it general/reusable

Worked example / how to — decompose an online cinema booking system

Break the problem into ordered sub-procedures:

1. `selectFilmAndShowtime()` — choose film, date and time.
 2. `checkSeatAvailability()` — depends on step 1's output (which showing).
 3. `chooseSeats()` — depends on step 2 (only offer free seats).
 4. `takePayment()` — depends on step 3 (must know the price/number of seats).
 5. `confirmBookingAndUpdateSeatMap()` — depends on step 4 (only confirm once paid).
- **Fixed order**: payment can't run before seats are chosen; the seat map can't update before payment succeeds — these **data dependencies** set the sequence.
 - Each routine does one job and could be reused (e.g. `takePayment()` on other pages).

Exam tips

- **Apply, don't define** — name the scenario's actual sub-procedures and put them in order.
- State **which steps depend on earlier outputs** to justify the sequence — this is what earns AO2 marks.
- Keep **decomposition** (split into steps) distinct from **abstraction** (remove detail).
- Note where a sub-procedure could be **reused**, linking to thinking ahead (2.1.2).